

Week03 | 采集与入湖——Batch / CDC / Stream 的组合拳

把 Week02 的 contract / manifest 变成真正可运行、可重跑、可补数的 ingestion baseline

这一周的重点不再是定义输入边界，而是把这些边界真正推进到采集链路里：

manifest 怎么声明一次 ingest、state 怎么记录批次进度、哪些数据该 accept / quarantine / reject、为什么 replay / backfill 必须被当成一等工程能力。

如果 Week02 解决的是“什么数据允许进入系统”，Week03 解决的就是：允许进入之后，这批数据到底怎样稳定进入 lake、怎样保留状态、怎样支持重跑与回放。

本周所有实践默认在 [OmniSupport Copilot 项目仓库](#) 中完成。课程讲义负责解释判断、结构与最小闭环，代码与验证以项目仓库当前结构为准。

Week03 默认优先对齐当前仓库现实路径：`contracts/data/*.json`、`data/seed_manifests/*.json`、`pipelines/ingestion/*.py`、`tests/contract/test_json_schemas.py`，命令默认采用 Docker-first 路径。

本周一句话

把 Week02 的 contract / manifest / gate，收成一条可运行、可重跑、可补数、可回放、可解释的采集与入湖基线。

本周为什么重要

- “采到数据”不等于链路可靠；真正危险的是数据偶尔能进、偶尔不进，而且没有稳定 state 和 run evidence。
- 如果 Week03 没把 ingestion baseline 做对，后面的检索、索引、生成、评测都会建立在不可复现的输入批次上。
- Batch、Incremental、CDC、Replay、Backfill 不是五个术语，而是五类不同的运行时承诺。
- 只有把 state、manifest、gate 和 report 接起来，系统才真正具备“今天能跑，明天还能解释为什么这样跑”的能力。

Week03 不是什么

- 不是一周把 Kafka、Debezium、流式编排和自动补数平台一次性搭完的工具秀。
- 不是“把数据先搬进来再说”的 ETL 演示；这周要先立住的是 runtime capability，而不是一次性跑通。
- 不是用新脚本再造一套练习世界；实践必须继续贴着 OmniSupport Copilot 当前 repo 的真实对象。
- 不是假装所有恢复能力都已经 fully automated；当前阶段先把 state、report、replay / backfill 边界和 runbook 写扎实。

本周在整门课里的位置

上承 Week02

接住资产盘点、最小元数据、PII 分级、Data Contract 和 source manifest 的定义。

本周解决

解决 ingest admission、批次状态、增量窗口、乱序、回放和补数的工程化落地。

下接 Week04

把落库后的资产、分层与可消费数据集，真正交给检索、索引和 Lakehouse 组织。

影响后续

Week08 的 RAG API、Week09 的工具链、Week11 之后的评测与治理，都会持续消费 Week03 定义的 state、report 和 replay 能力。

五个课时

Week03 | 课时 1 | 从“能采上来”到“可重复采集”：为什么 ingest 可靠性决定下游一切

先立住判断：很多系统不是“采不到”，而是“今天能采、明天歪了、后天又补不回来”。

- 输入批次为什么会破坏可复现性
- contract 与 manifest 为什么还不足以保证 ingestion 稳定
- state、run evidence 和 replay 为什么必须前置

进入课时 1

Week03 | 课时 2 | 批量采集主链路：幂等写入、重跑设计与完整性校验

先把最稳的 Batch 路径做对，再谈更复杂的增量与流式。

- manifest 怎样驱动 batch ingest
- 结构化和文档采集怎样建立最小重跑基线
- run report 和 batch state 为什么是第一批运行时证据

进入课时 2

Week03 | 课时 3 | 增量与 CDC：cursor、WAL、乱序、去重与“不要轻易承诺 exactly-once”

把“每天跑一次”升级成真正可控的增量系统。

- cursor 和 watermark 的边界
- 去重、迟到数据和窗口错位怎么处理
- 为什么 state 一旦写错，补数和回放都会失真

进入课时 3

Week03 | 课时 4 | 从任务流到资产流：用 Dagster 组织 ingest、分区与回放

这节课不追求把 Dagster 讲成平台百科，而是先把 ingest 资产化所需要的关键概念立住。

- asset / materialization / partition / backfill / asset check 各自负责什么
- 为什么 ingest 一旦被看成资产流，Week04 / Week06 的回放与补数会更稳
- 当前 repo 的 `assets.py` 和 `definitions.py` 已经给了你哪些最小骨架

进入课时 4

Week03 | 课时 5 | 故障自愈与补数：Replay / Backfill / Runbook 怎么把链路拉回正轨

Week03 的收官不是“系统终于跑通”，而是故障发生后你知道怎样把链路拉回正轨。

- retry / rerun / replay / restore / backfill 的边界
- runbook 为什么是 Week03 就该出现的工程资产

- 哪些恢复动作依赖 contract / manifest / state / run log 一起判断

进入课时 5

实验 | 双源采集最小闭环:ticket + document ingest、state、report、replay

实验目标是跑出一条可验证的最小 ingestion baseline:

- 用 ticket + document 两类 source 做一次双源 ingest
- 生成 state 和 run report
- 模拟一次 replay 或 backfill
- 观察 manifest、state 和 run evidence 如何共同决定“这批数据算不算可靠落地”

进入实验

作业 | 采集最小链路 v1、Runbook v1 与完整性报告

本周作业要求学员把采集路径整理成正式提交包:

- 一份 Week03 ingest baseline 说明
- 一条双源最小采集链路
- 一份 runbook v1
- 一份完整性报告, 说明 state、report、replay 的判断依据

进入作业

本周交付物

- docs/blueprints/week03/ingestion_baseline_v1.md
- docs/blueprints/week03/checkpoint_state_v1.md
- runbooks/ingestion_runbook_v1.md
- reports/week03/recovery_drill_report.md
- 一组 state / report / replay / backfill 观察记录

关键判断

- ingest 成功一次, 不等于 ingest 能稳定复现。
- manifest 决定“这次接什么”, state 决定“接到哪里”, report 决定“到底发生了什么”。
- replay 和 backfill 不是补丁动作, 而是 ingestion baseline 的内建能力。
- Week03 做得越扎实, Week04 之后的索引、检索、引用和治理越稳。

上下游关系

- 上承 Week02: 把 contract、manifest、gate 变成真正运行时入口。
- 下接 Week04: 把 ingest 结果变成可资产化、可索引、可消费的数据层。
- 影响 Week08–Week14: state、trace、run evidence、replay 和 release 记录会持续进入后面的治理与评测闭环。